



Git it done!

Introduction to version control with *git*

Mario Sánchez Prada
msanchez@igalia.com

A Coruña. 22th of July, 2010

“I think Git overshadows any other achievement that Linus guy ever had. He should stop working on that hobby kernel, and put more time into Git, I think it has potential.”

*Eitan Isaacson
Open Source developer
October 21st, 2009*

About me

- Computer Science Engineer by the *University of Coruña*
- Member of the Free Software company *Igalia*
- Free Software developer (*GNOME & Maemo*)

About me

- Computer Science Engineer by the *University of Coruña*
- Member of the Free Software company *Igalia*
- Free Software developer (*GNOME* & *Maemo*)
- *git* user since **July 2008**

- **Distributed** Version Control System (*DVCS*)
- Initially written in C by Linus Torvalds
- Replacement for *BitKeeper* in Linux kernel development
- Widely used nowadays, both for new and old projects

`http://git-scm.com`

Basic concepts

The problem

Handling changes in a file, a project...

The problem

Handling changes in a file, a project...

- Option 0: Do nothing (unique version)

The problem

Handling changes in a file, a project...

- Option 0: Do nothing (unique version)
- Option 1: Rudimentary system (manual copies)
 - Project
 - Project.bckp
 - Project-20100514.bckp
 - Project-20100514.bckp2
 - Project-20100514.bckp2-final
 - Project-20100514.bckp2-last
 - Project-20100514.bckp2-last-final
 - Project-????????

The problem

Handling changes in a file, a project...

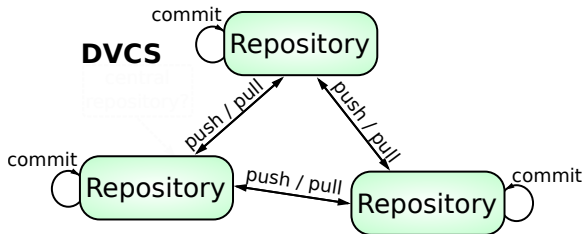
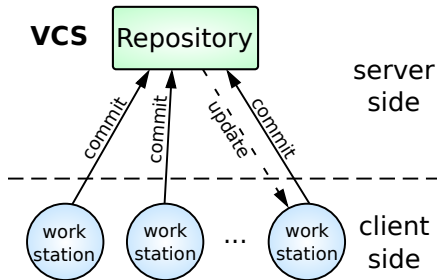
- Option 0: Do nothing (unique version)
- Option 1: Rudimentary system (manual copies)
 - Project
 - Project.bckp
 - Project-20100514.bckp
 - Project-20100514.bckp2
 - Project-20100514.bckp2-final
 - Project-20100514.bckp2-last
 - Project-20100514.bckp2-last-final
 - Project-???????
- Option 2: “Traditional” VCS’s: *CVS*, *Subversion*...
 - They normally require a connection to a central repository

The problem

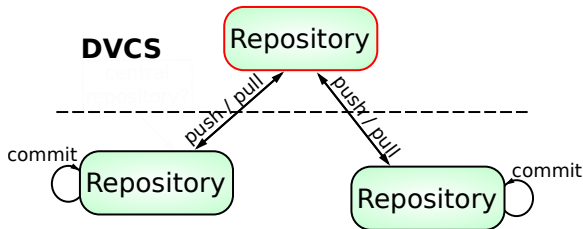
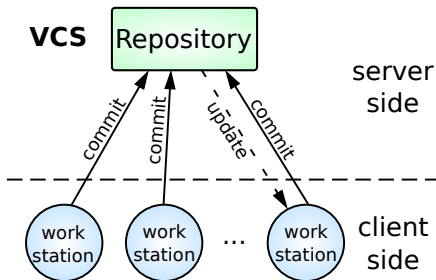
Handling changes in a file, a project...

- Option 0: Do nothing (unique version)
- Option 1: Rudimentary system (manual copies)
 - Project
 - Project.bckp
 - Project-20100514.bckp
 - Project-20100514.bckp2
 - Project-20100514.bckp2-final
 - Project-20100514.bckp2-last
 - Project-20100514.bckp2-last-final
 - Project-???????
- Option 2: “Traditional” VCS’s: *CVS*, *Subversion*...
 - They normally require a connection to a central repository
- Option 3: *DVCS*’s: *Bazaar*, *Mercurial*, *Monotone*, ***git*** ...
 - **Total** autonomy and flexibility (i.e. local *commits*)

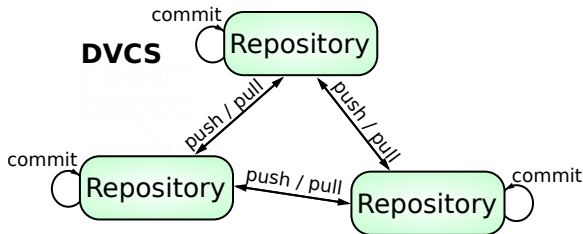
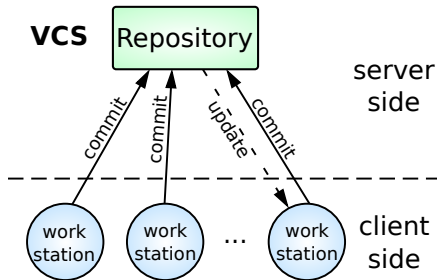
VCS vs DVCS



VCS vs DVCS



VCS vs DVCS



Advantages of DVCS

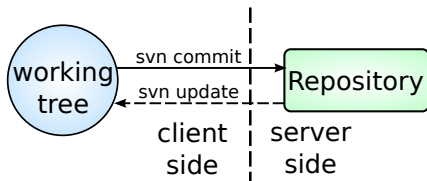
- Full repository in the local machine: new possibilities: local history (full log), branches management, offline *commits*...
- Improve the *workflow* for collaborative work
- Make integration of changes from external sources easier
- Make handling different versions for a project easier
- Better separation between private and public work
- Implicit back-up copies
- Don't need network connection for most of the operations

Disadvantages of DVCS

- More complex than *VCS's*. Harder learning curve
- Initially cloning is slower (full repository cloning)
- Full clones can require more hard disk space than a single checkout of the last revision (like in SVN)

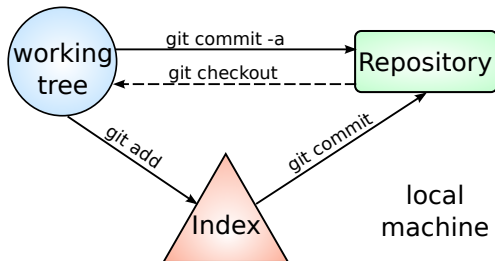
Subversion (VCS) vs Git (DVCS)

Subversion



Requires the two client / server parts to work

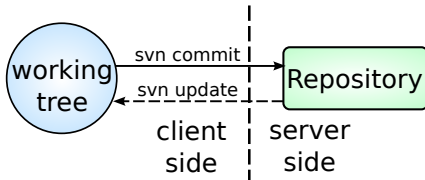
Git



Doesn't require anything else but the local machine itself

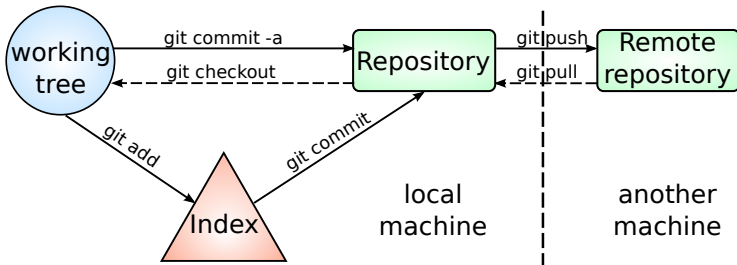
Subversion (VCS) vs Git (DVCS)

Subversion

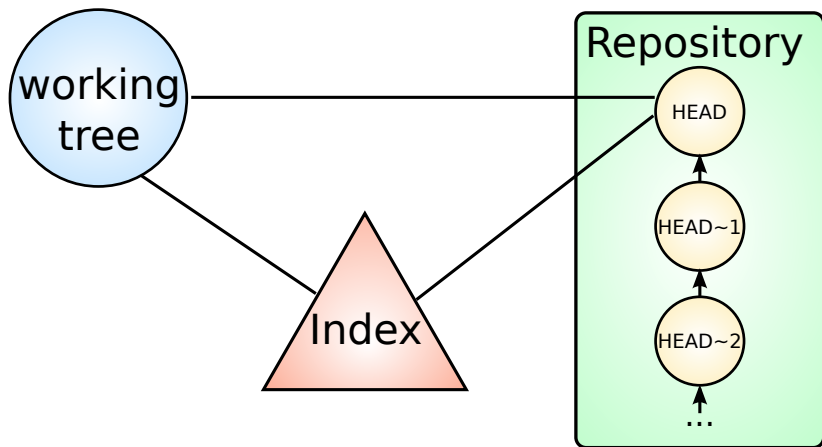


Requires the two client / server parts to work

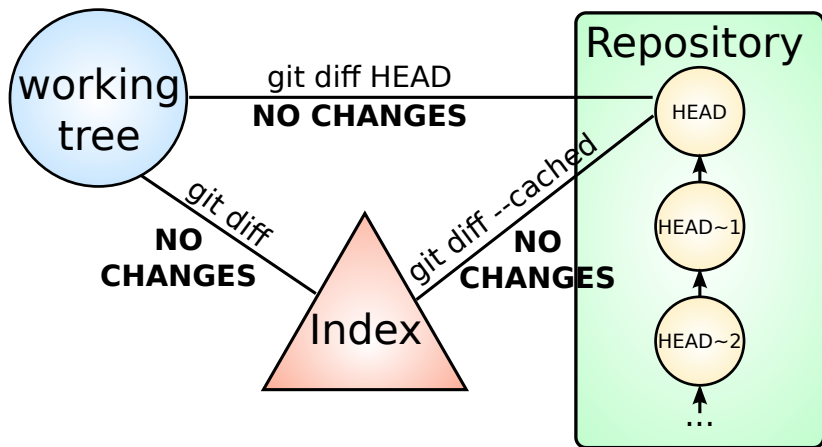
Git



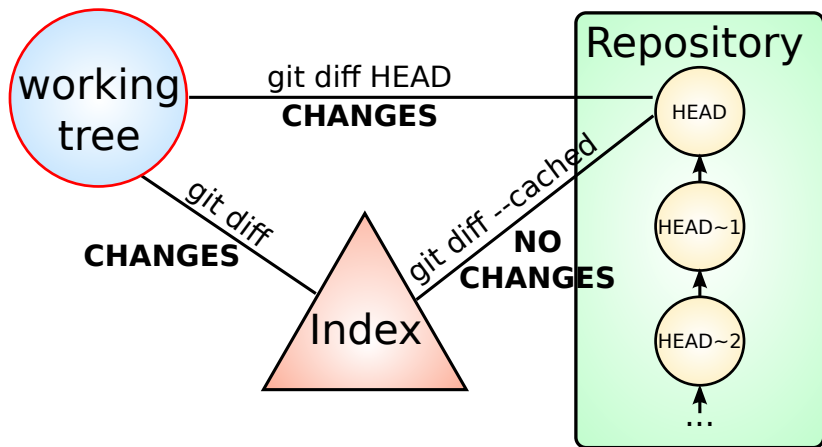
Typical workflow in git



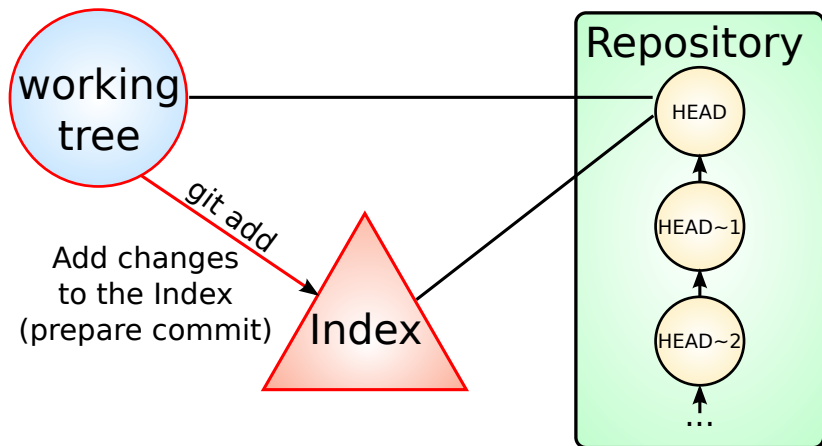
Typical workflow in git



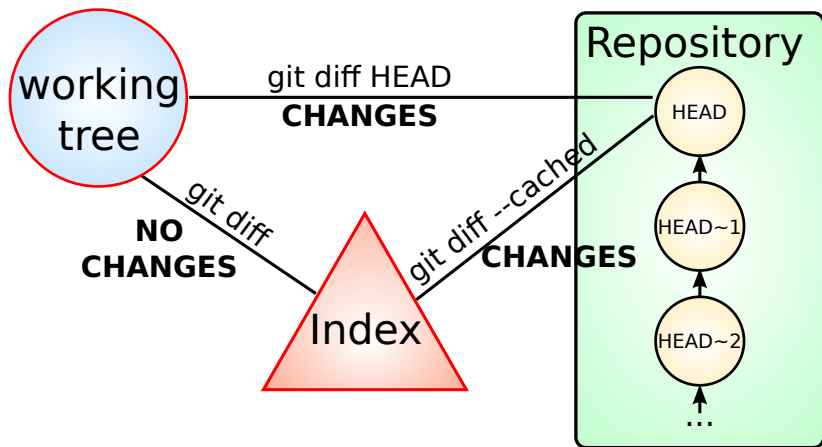
Typical workflow in git



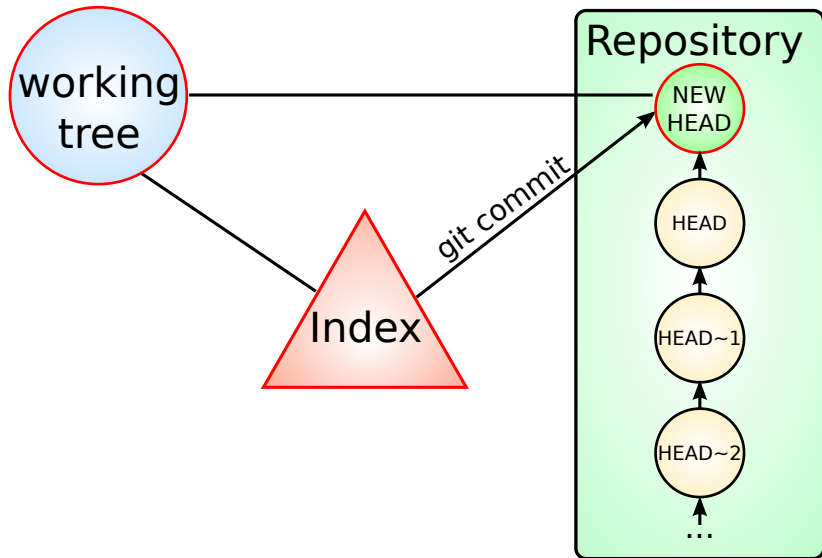
Typical workflow in git



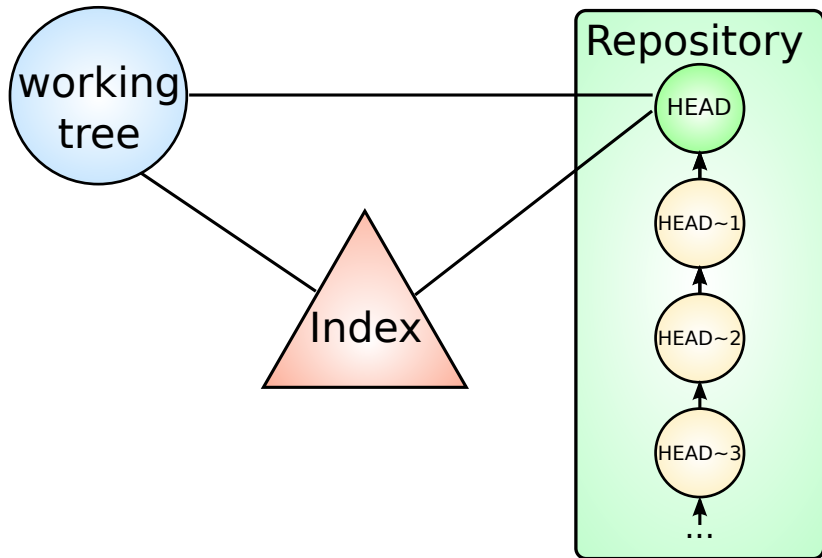
Typical workflow in git



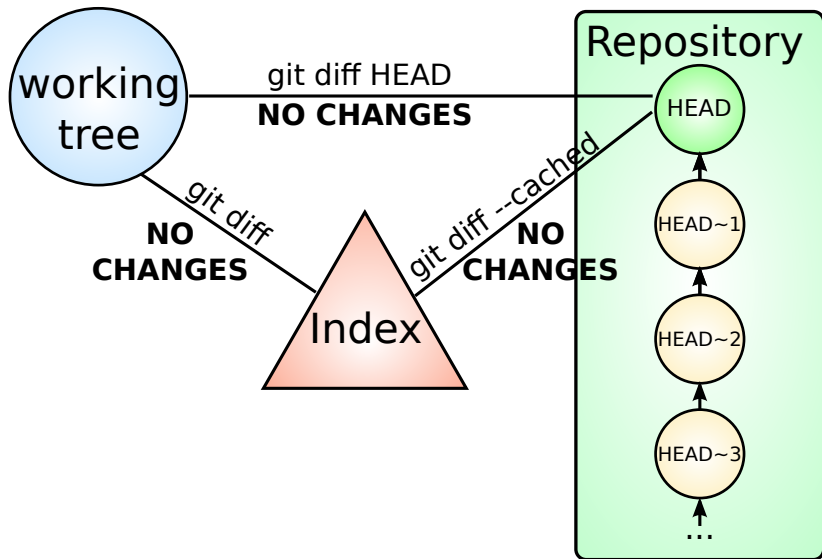
Typical workflow in git



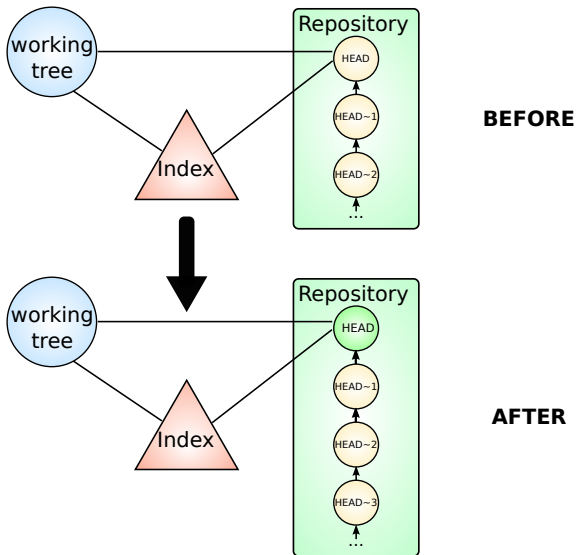
Typical workflow in git



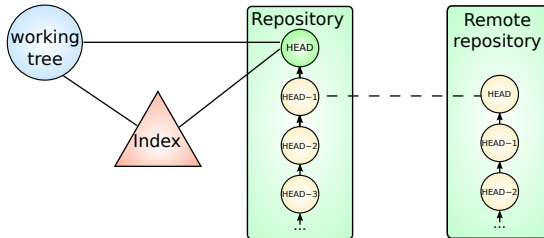
Typical workflow in git



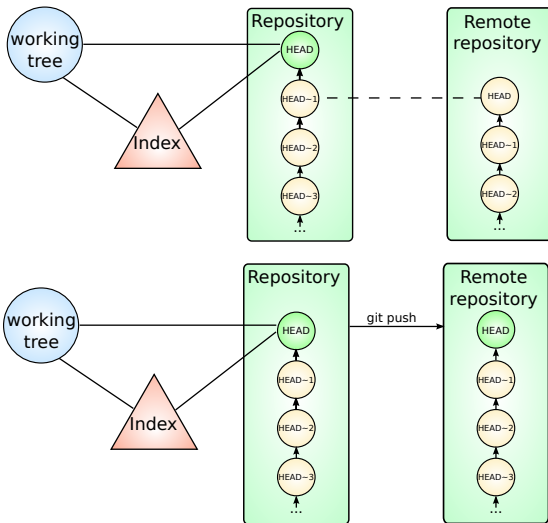
Typical workflow in git



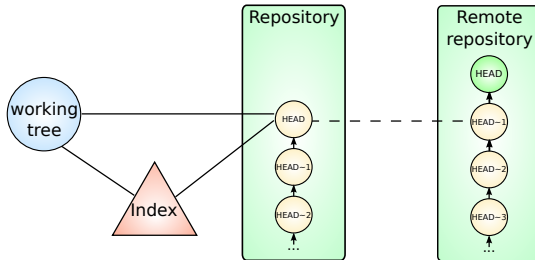
Typical workflow in git - git push



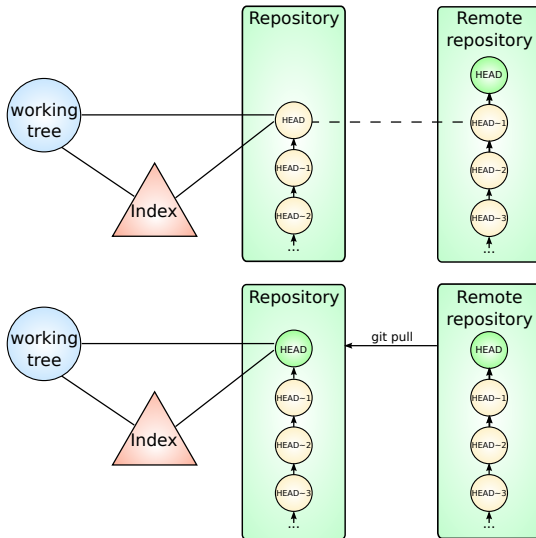
Typical workflow in git - git push



Typical workflow in git - git pull



Typical workflow in git - git pull



Typical workflow in git - an example

```
$ mkdir /tmp/git
```

```
$ cd /tmp/git
```

```
$ git init . # Initialize repository
```

```
Initialized empty Git repository in /tmp/git/.git/
```

```
$ touch A B C
```

```
$ git add A B C # Add new files to Index
```

```
$ git commit -m "Initial commit" # First commit!
```

```
[master (root-commit) fal6ae4] Initial commit
```

```
0 files changed, 0 insertions(+), 0 deletions(-)
```

```
create mode 100644 A
```

```
create mode 100644 B
```

```
create mode 100644 C
```


Typical workflow in git - an example

```
$ git diff --stat          # Working Tree <-> Index
$ git diff --stat HEAD    # Working Tree <-> HEAD
$ git diff --stat --cached # Index <-> HEAD
```

<<< MAKE SOME CHANGES IN THE WORKING TREE >>>

```
$ git diff --stat          # Working Tree <-> Index
A |      3 +++
C |      3 +++
2 files changed, 3 insertions(+), 0 deletions(-)
```

```
$ git diff --stat HEAD    # Working Tree <-> HEAD
A |      3 +++
C |      3 +++
2 files changed, 3 insertions(+), 0 deletions(-)
```

```
$ git diff --stat --cached # Index <-> HEAD
```

Typical workflow in git - an example

```
$ git add A C                                # Move changes to Index

$ git diff --stat                             # Working Tree <-> Index

$ git diff --stat HEAD                       # Working Tree <-> HEAD
A |      3 +++
C |      3 +++
2 files changed, 3 insertions(+), 0 deletions(-)

$ git diff --stat --cached                   # Index <-> HEAD
A |      3 +++
C |      3 +++
2 files changed, 3 insertions(+), 0 deletions(-)
```

Typical workflow in git - an example

```
$ git commit -m "My first patch :-)"    # Commit changes  
[master 614c4e2] My first patch :-)  
2 files changed, 3 insertions(+), 0 deletions(-)
```

```
$ git diff --stat                      # Working Tree <-> Index  
$ git diff --stat HEAD                 # Working Tree <-> HEAD  
$ git diff --stat --cached             # Index <-> HEAD
```

```
$ git show --stat HEAD                 # Show last commit's stats  
commit 614c4e224284e2719fe040b64685b790606000a6  
Author: Mario Sanchez Prada <msanchez@igalia.com>  
Date:   Mon May 3 09:31:19 2010 +0200
```

```
My first patch :-)
```

```
A |      3 +++  
C |      3 +++  
2 files changed, 3 insertions(+), 0 deletions(-)
```

Typical workflow in git - an example

```
$ git log                                # Show history up-to-date
commit 614c4e224284e2719fe040b64685b790606000a6
Author: Mario Sanchez Prada <msanchez@igalia.com>
Date:   Mon May 3 09:31:19 2010 +0200
```

My first patch :-)

```
commit fal6ae4ff64b3b968ca8991c90f123e62a6a009c
Author: Mario Sanchez Prada <msanchez@igalia.com>
Date:   Mon May 3 09:30:11 2010 +0200
```

Initial commit

Typical workflow in git (wrapping up)

Basic commands:

- `git init`: Initializes the repository to work with *git*
- `git add`: Adds changes in the *Working Tree* to the *Index*
- `git commit`: Makes changes in the *Index* permanent, creating a new commit and updating the *HEAD*.
- `git diff`: Differences between *Working Tree* and *Index*
- `git push`: Sends local commits to a remote repository
- `git pull`: Brings commits from a remote repository

Typical workflow in git (wrapping up)

Basic commands (extended version):

- `git init`: Initializes the repository to work with *git*
- `git add`: Adds changes in the *Working Tree* to the *Index*
 - `git add -u`: Adds all the changes to the *Index*
 - `git add -patch`: Interactively asks which changes should go to the *Index* (*hunk by hunk*)
- `git commit`: Makes changes in the *Index* permanent, creating a new commit and updating the *HEAD*.
 - `git commit -a:git add -u && git commit`
- `git diff`: Differences between *Working Tree* and *Index*
 - `git diff HEAD`: Diff between *Working Tree* and *HEAD*
 - `git diff -cached`: Diff between *Index* and *HEAD*
- `git push`: Sends local commits to a remote repository
- `git pull`: Brings commits from a remote repository

Typical workflow in git (wrapping up)

Other (extremely useful) commands:

- `git clone <URL>`: Clones a remote repository
- `git rm <file>`: Removes *file* and get the *Index* ready with the changes for the next *commit*.
- `git mv <fileA> <fileB>`: Renames *fileA* as *fileB* and get the *Index* ready with the changes.
- `git log`: Shows the full (*log*) from *HEAD*
- `git status`: Shows the status of *Working Tree* and *Index*
- `git show <commitID>`: Visualizes *commit* *commitID*

Typical workflow in git (wrapping up): git status

```
<<< RIGHT BEFORE ADDING A AND C TO INDEX WITH 'git add A C' >>>
```

```
$ git status
# On branch master
# Changed but not updated:
#   (use "git add <file>..." to update what will be committed)
#   (use "git checkout -- <file>..." to discard changes in working directory)
#
# modified:   A
# modified:   C
#
no changes added to commit (use "git add" and/or "git commit -a")
```

```
$ git add A
```

```
$ git status
# On branch master
# Changes to be committed:
#   (use "git reset HEAD <file>..." to unstage)
#
# modified:   A
#
# Changed but not updated:
#   (use "git add <file>..." to update what will be committed)
#   (use "git checkout -- <file>..." to discard changes in working directory)
#
# modified:   C
#
```


Typical workflow in git (wrapping up): git show

```
<<< RIGHT AFTER COMMITTING CHANGES WITH 'git commit' >>>
```

```
$ git show
commit 614c4e224284e2719fe040b64685b790606000a6
Author: Mario Sanchez Prada <msanchez@igalia.com>
Date:   Mon May 3 09:31:19 2010 +0200
```

```
    My first patch :-)
```

```
diff --git a/A b/A
index e69de29..1802a74 100644
--- a/A
+++ b/A
@@ -0,0 +1,3 @@
+aaa
+bbb
+ccc
diff --git a/C b/C
index e69de29..1802a74 100644
--- a/C
+++ b/C
@@ -0,0 +1,3 @@
+aaa
+bbb
+ccc
```

Comparing commands: Subversion - Git

Subversion	Git
svnadmin create <i>repo</i> svn import <i>filepath</i>	git init git add <i>filepath</i> git commit
svn checkout <i>URL</i>	git clone <i>URL</i>
svn add rm mv <i>file</i>	git add rm mv <i>file</i>
svn diff	git diff
svn commit	git commit -a git push
svn update	git pull
svn status	git status
svn log	git log
svn blame <i>file</i>	git blame <i>file</i>

Repositories and branches

Repositories

Two types of repositories:

- **Local repository:** Local working directory under version control with *git* (it contains a `.git/` directory)

They can be “created” by either:

- `git init`: Initializes the current directory to be used with *git* (needed to `git add <file>` and `git commit`)
 - `git clone <URL>`: Clones a remote repository in a local directory, establishing a link to the original one, throughout the remote repository `origin`.
- **Remote repositories:** Any non-local repository where we have cloned our local working directory from

Can be linked with `git remote add <name> <URL>`
`git clone <URL>` automatically creates one: `origin`

What's a branch?

A *branch* is a parallel line of work which allow us to make and try out certain changes from a given point on, without altering the original line of work it was created from.

What's a branch?

A *branch* is a parallel line of work which allow us to make and try out certain changes from a given point on, without altering the original line of work it was created from.

Once the changes done in the *branch* have been proved to work well, they can be integrated in the original *branch*, resolving the “conflicts” that could have appeared since its creation.

What's a branch?

A *branch* is a parallel line of work which allow us to make and try out certain changes from a given point on, without altering the original line of work it was created from.

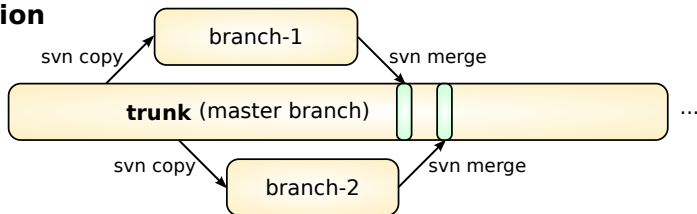
Once the changes done in the *branch* have been proved to work well, they can be integrated in the original *branch*, resolving the “conflicts” that could have appeared since its creation.

Implementation:

- **In other VCS's (*Subversion*):** Separate directories
- **En *git* :** An *alias* to the *commit* that will be the *HEAD* for that *branch* (almost-zero cost, really fast).

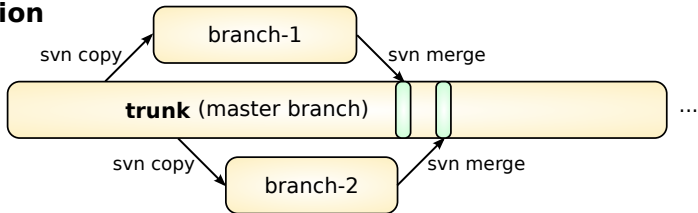
What's a branch? - Subversion vs Git

Subversion

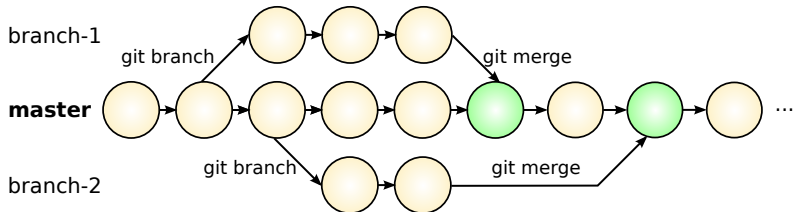


What's a branch? - Subversion vs Git

Subversion



Git



Branches in git

- Create a new branch:

```
git branch <branch-name> [<start-point>]
```

- Delete a branch:

```
git branch (-d | -D) <branch-name>
```

- Rename a branch:

```
git branch -m <old-name> <new-name>
```

- List the current branches:

```
git branch [ -r | -a ]
```

- “Move” into a branch (check it out):

```
git checkout [-b] <branch-name>
```

Branches in git - an example

```
$ git branch
```

```
* master
```

```
$ git branch mybranch-1
```

```
$ git branch mybranch-2
```

```
$ git branch
```

```
* master
```

```
mybranch-1
```

```
mybranch-2
```

```
$ git checkout mybranch-1
```

```
Switched to branch 'mybranch-1'
```

```
$ git branch
```

```
master
```

```
* mybranch-1
```

```
mybranch-2
```

Branches in git - an example

```
$ git branch -m mybranch-1 branch-1
```

```
$ git branch -m mybranch-2 branch-2
```

```
$ git branch
```

```
master
```

```
* branch-1
```

```
branch-2
```

```
$ git checkout master
```

```
Switched to branch 'master'
```

```
$ git branch -d branch-1
```

```
Deleted branch branch-1 (was 59f1309).
```

```
$ git branch -d branch-2
```

```
Deleted branch branch-2 (was e721bce).
```

```
$ git branch
```

```
* master
```

Why to use branches?

- They are fast, convenient, and really “cheap”
- To try an idea out (experiment)
- Separate public and private work
- One branch per *bug/feature*
- Prepare for a new *release* (integration)
- As a temporary *backup (snapshot)*
- **It's the usual way of working with *git***

Local branches and remote branches

Two types of branches:

- **Local branches:** Branches created in the local machine to work on top of them, making all the needed changes.
- **Remote branches:** Branches from a remote repository that we can link to local branches to work with them.

Clarifications:

Both the **local** and the **remote** branches are always in our **local repository** (local/remote concept depends on the POV).

The **remote** branches act as a *cache* of non-local branches in **remote repositories** (they are *rsync-ed*).

The **local** branches are *pushed/pulled* to/from the **remote repositories** throughout their associated **remote** branches.

Local branches and remote branches - an example

```
$ git branch -a
* master
  remotes/origin/master
  remotes/origin/other-branch
```

```
$ git branch private
```

```
$ git branch other-branch origin/other-branch
Branch other-branch set up to track
remote branch other-branch from origin.
```

```
$ git branch -a
* master
  private
  other-branch
  remotes/origin/master
  remotes/origin/other-branch
```

An example of repositories and branches: epiphany

Local branches

```
$ git branch
136292
539716
602547
608980
* 611400
alex
master
```

Using branches in this example:

- One branch per *bug*
- One experimental branch (*alex*)
- One “main” branch (*master*).

Remote branches

```
$ git remote
origin

$ git branch -r
origin/HEAD -> origin/master
origin/Release043
origin/bookmarks-toolbars-changes
origin/bookmarksbar-separation
origin/css-fonts-branch
origin/eog-menu-api
origin/gnome-2-10
[...]
origin/master
origin/mozilla-embed-strings
origin/new-completion
origin/new-downloader
origin/pre-gnome-2-10
origin/pre-gnome-2-14
origin/pre-gnome-2-6
origin/pre-gnome-2-8
origin/release-1-2-1
origin/webcore-branch
origin/webkit
origin/xulrunner
```


Updating the branches

Two typical situations:

- **Add** changes in a **private branch** to a **public branch**
(integrate local changes)

- **Modify** a **private branch** with new code from a **public branch** where it was created from (update)

Updating the branches

Two typical situations:

- **Add** changes in a **private branch** to a **public branch** (integrate local changes)
 - We can't alter past history in a public branch
 - We need “clean” changes ready to be applied on top of *HEAD* in the public branch
- **Modify** a **private branch** with new code from a **public branch** where it was created from (update)

Updating the branches

Two typical situations:

- **Add changes in a private branch to a public branch** (integrate local changes)
 - We can't alter past history in a public branch
 - We need “clean” changes ready to be applied on top of *HEAD* in the public branch
- **Modify a private branch with new code from a public branch** where it was created from (update)
 - We can alter past history in a private branch
 - It's more interesting to “change the past” whenever it's needed in order to always have “clean changes” in the branch where it was created from (it could have changed)

Updating branches - git merge

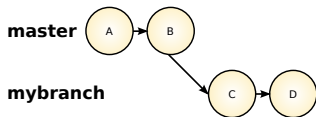
Updating a public branch: `git merge <branch>`

- There are **no changes** in the public branch where the private one was created from:
- There are **changes** in the public branch where the private one was created from:

Updating branches - git merge

Updating a public branch: `git merge <branch>`

- There are **no changes** in the public branch where the private one was created from:

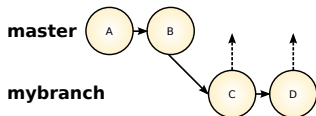


- There are **changes** in the public branch where the private one was created from:

Updating branches - git merge

Updating a **public branch**: `git merge <branch>`

- There are **no changes** in the public branch where the private one was created from:

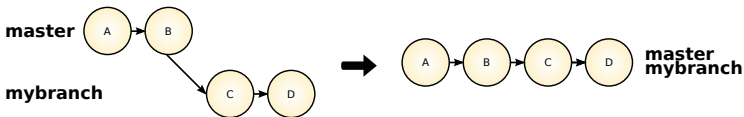


- There are **changes** in the public branch where the private one was created from:

Updating branches - git merge

Updating a public branch: `git merge <branch>`

- There are **no changes** in the public branch where the private one was created from:

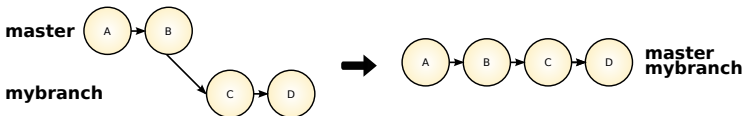


- There are **changes** in the public branch where the private one was created from:

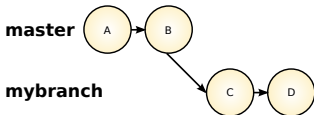
Updating branches - git merge

Updating a public branch: `git merge <branch>`

- There are **no changes** in the public branch where the private one was created from:



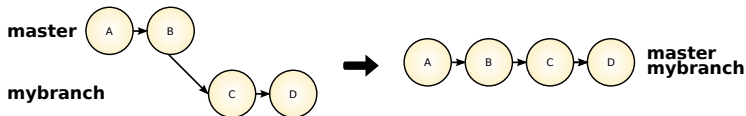
- There are **changes** in the public branch where the private one was created from:



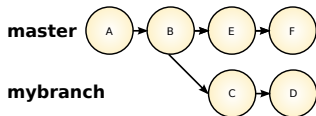
Updating branches - git merge

Updating a public branch: `git merge <branch>`

- There are **no changes** in the public branch where the private one was created from:



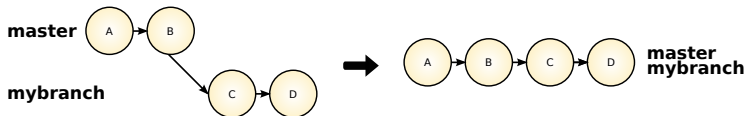
- There are **changes** in the public branch where the private one was created from:



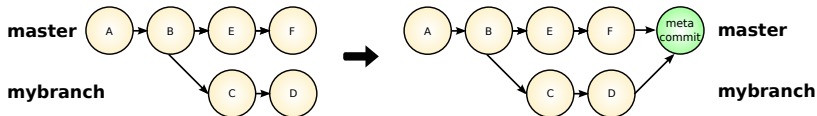
Updating branches - git merge

Updating a public branch: `git merge <branch>`

- There are **no changes** in the public branch where the private one was created from:

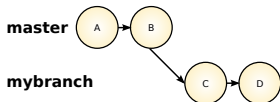


- There are **changes** in the public branch where the private one was created from:



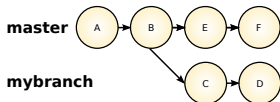
Updating branches - git rebase

Update a private branch: `git rebase <new-base>`



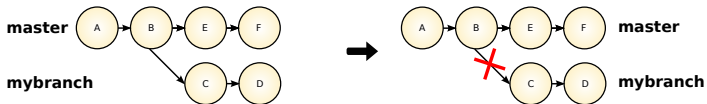
Updating branches - git rebase

Update a private branch: `git rebase <new-base>`



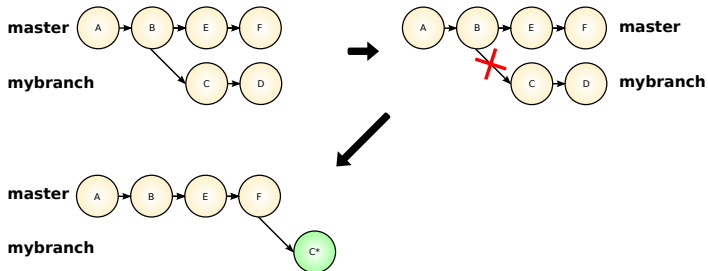
Updating branches - git rebase

Update a private branch: `git rebase <new-base>`



Updating branches - git rebase

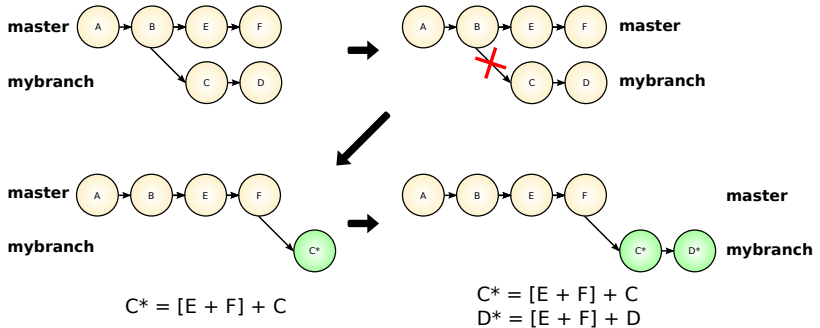
Update a private branch: `git rebase <new-base>`



$$C^* = [E + F] + C$$

Updating branches - git rebase

Update a private branch: `git rebase <new-base>`



One personal tip (whenever it's possible)

Before updating a public branch with `git merge` to do a `git push` later on, it's a good idea to do a `git rebase` first from the private branch, in order to avoid getting a *metacommit*.

```
$ git checkout mybranch # Change to private branch  
  
$ git rebase master      # Rebase private branch  
  
$ git checkout master    # Change back to master  
  
$ git merge mybranch     # Merge changes from mybranch  
  
$ git push               # Push to remote repository
```


Other things...

Other interesting commands

- Create a tag:
`git tag <tag-name>`
- Revert a *commit* (create a new *commit* on top of *HEAD*):
`git revert <commitID>`
- Copy a specific commit on top of *HEAD*:
`git cherry-pick <commitID>`
- Show the authors of the latest changes to a file:
`git blame <filepath>`
- Apply *diff* in the *Working Tree* (doesn't create new *commits*):
`git apply <diff-file>`
- Create “full” patches from a given point (commit):
`git format-patch <base-commitID>`
- Apply “full” patches (keeping the author, log message...):
`git am <patchfile>`

For further investigation

- Save and restore temporary changes:
`git stash` & `git stash apply`
- Modifying *HEAD*, *Index* and the *Working Tree*:
`git reset [ -soft | -hard ] <commitID>`
- Manually rewriting the past (*commits*):
 - `git rebase -i <commitID>`
 - `git commit -amend`
- Automatically rewriting the past:
`git filter-branch <params>`
- Check out the history of git operations (and restore data):
`git reflog`
- Send patches by mail:
`git send-email <email-address>`
- Integration with *Subversion* repositories:
`git svn <command> [ options ] [ arguments ]`

Where to go from here

- ***Git from the bottom up (Excellent tutorial):***
<http://ftp.newartisans.com/pub/git.from.bottom.up.pdf>
- ***Git User's Manual:***
<http://www.kernel.org/pub/software/scm/git/docs/v1.7.1/user-manual.html>
- ***Git tutorial manual page (man gittutorial):***
<http://www.kernel.org/pub/software/scm/git/docs/v1.7.1/gittutorial.html>
- ***Everyday GIT With 20 Commands Or So:***
<http://www.kernel.org/pub/software/scm/git/docs/v1.7.1/everyday.html>
- ***Gitorious: infrastructure for open source projects with Git:***
<http://gitorious.org>

Questions?

Thank you!